Anteater Chess Ver1.0

Software Architecture Specification

April 13, 2026

The Knight Owls: Akshithi Suresh, Jolynn Quang, Jasmine Kong, Troy Reindl, Ayda Ahmadian, Trisha Satyavrat

EECS 22L, UC Irvine

Table of Contents

Glossary

Board - The 10x8 grid on which Anteater Chess is played, with columns labeled A–J and rows labeled 1–8.

File - A column on the board, labeled A through J.

Rank - A row on the board, numbered 1 through 8.

White/Black - The two sides in Anteater Chess. White always moves first.

Ant (Pawn) - A piece that can move 1-2 spaces straight forward on its first turn and 1 space forward on every turn after. The ant captures diagonally.

Anteater - A special piece unique to Anteater Chess. Moves one square in any direction like a King. When capturing, the Anteater can eat multiple ants (pawns) in a single move, as long as they are adjacent to each other horizontally or vertically (not diagonally)

Ant eating - a special move that allows the anteater piece to continuously capture ants (pawns) who are adjacent to each other in the same file or rank but not diagonally

Bishop - A piece that moves diagonally any number of times.

King - The most important piece on the board. Moves one square in any direction. If the King is checkmated, the game is over.

Knight - A piece that moves in an L-shape: two squares in one direction and one square perpendicular. The only piece that can jump over other pieces.

Queen - The most powerful piece on the board. Can move any number of squares horizontally, vertically, or diagonally.

Rook - A piece that moves any number of squares horizontally or vertically.

Capture - The act of moving a piece onto a square occupied by an opponent's piece, removing it from the board.

Castling - A special move involving the king and rook. It is allowed only if neither piece has moved, the squares between them are empty, the king is not in check, and the king does not move through or end on a square under attack.

Check - A situation where a player's King is under direct threat of capture on the opponent's next move. The player must resolve the check immediately.

Checkmate - A situation where a player's King is in check, and there is no legal move to escape. The game ends, and that player loses.

En Passant - A special pawn capture that can occur immediately after an opponent's pawn moves two squares forward from its starting position. The capturing pawn moves to the square the opponent's pawn passed through.

Legal Move - Any move that follows the rules of Anteater Chess and does not leave the player's own King in check.

Stalemate - A situation where a player has no legal moves available, but their King is not in check. The game ends with no winner.

API – The shared interface that allows different parts of the program to talk to each other by calling each other's functions.

Coordinate – A specific position on the board identified by its file and rank (for example, F2).

Move – A data structure that records a single action in the game, storing the starting position and the destination position of a piece.

Game State – A snapshot of everything happening in the game at a given moment, including the board layout, whose turn it is, and the history of moves made so far.

Module – A self-contained unit of the codebase, typically a single .c file, that is responsible for one specific aspect of the program (such as input handling, rendering, or AI logic).

Validation – The process of checking whether a proposed move is legal according to the rules of the game before allowing it to be played.

Log File – A plain text file that records every move made during a game, preserving a complete history that can be reviewed later.

1. Software Architecture Overview

1.1 Main data types and structures

Chess Board:
Data Type: Board [8][10]
Function: Used to represent the board and its coordinate system
Implementation Logistics:
-Use a pointer to store a piece at an array index and null to represent an empty space

Piece Types:
Data Type: enum PieceType
Function: Used to properly identify a piece to determine possible moves and behaviors of pieces

Piece Color:
Data Type: enum PieceColor
Function: Define a piece as black or white

Piece:
Data Type: struct Piece
Function: Used to define each piece through piecetype and color

Move:
Data Type: struct Move
Purpose: Represent a move from one position to another
Implementation Logistics:
-From Pos1 to Pos2
-Pos1 represents start position while Pos2 represents end position

Pos:
Data Type: struct Pos
Purpose: Represent a board coordinate, for indexing pieces and moves
Implementation Logistics:
- Unsigned char r,f; (rank/file)

Move History (MoveLog):

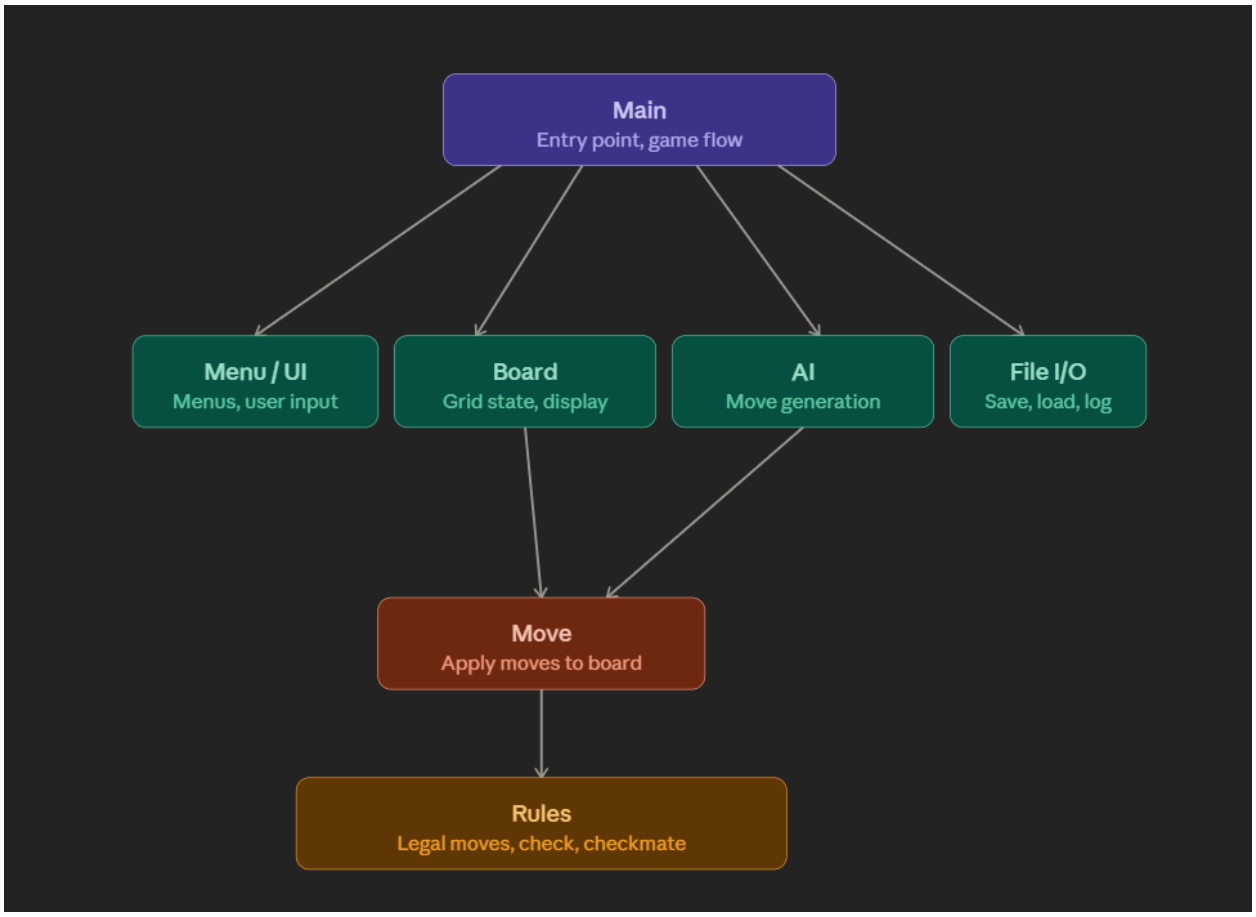
Data Type: struct MoveLog
Purpose: Stores the complete history of moves made during the game. Required for Undo, log file
output, en passant detection, and AI evaluation.

```
#define MAX_MOVES 512
struct MoveLog {
struct Move moves[MAX_MOVES];
int count;
};
```

Game State:
Data Type: struct GameState
Purpose: A snapshot of everything happening in the game at a given moment. All major functions
operate on a GameState. Used for save/load, AI, and undo.

```
struct GameState {
struct Piece *board[8][10];
enum PieceColor current_turn;
struct MoveLog history;
int white_castle_k;
int white_castle_q;
int black_castle_k;
int black_castle_q;
struct Pos en_passant_target;
};
```

1.2 Major Software Components



Anteater Chess is split into several modules, each handling a distinct part of the program.

Main Module:
- This controls the whole program.
- It starts the game, shows the menu, and runs the game loop.

Board Module:
Functions:
- Stores the chessboard.
- Board Display Function: Displays the board on the screen.
- Updates the board after every move.

Move Module:
Functions:
- Piece Move Function: Handles moving pieces from one position to another.

- Updates piece positions on the board.

Rules Module:
Functions:
- Legal Move Check Function: Checks if a move is allowed or not.
- Make sure all chess rules are followed.
- Handles special moves like castling, en passant, promotion, and anteater moves.
- King Check and Checkmate Function: Checks if a king is in check or checkmate.

AI Module:
- Controls the computer player.
- Compute all Legal Moves Function: Generates possible moves.
- Select Best Move Function: Picks one move to play.

File I/O Module:
Functions:
- Save Function: Saves the game to a file.
- Load Function: Loads a saved game.
- Log File Function: Writes moves into a log file.

Menu/UI Module:
Functions:
- Menu Function: Shows menu options to the user.
    - Takes user input (start, pause, quit, etc.).

1.3 Module Interfaces (API of Major Functions)

Board Module Interface:
- **Board Display Function**-Displays the chessboard and pieces
- **Look Up a Piece Function**-Checks which piece is located at a given position
- **Assign Piece Function**- Assigns a specific piece to a specific position

Menu/UI Module Interface:
- **Menu Function-** Shows menu options to the user
- **Paused Menu Function**- Pauses the game, allowing for saving, quitting, resuming, etc.

Move Module Interface:
- **Piece Move Function-** Moves the position from the starting position to a second position
- **Compute All Reachable Moves Function-** Determine what positions a piece can move to based on the movement pattern of the piece

Rules Module Interface:

- **Compute all legal moves function-** Determine all positions a piece can move within the rules of Anteater chess
- **Legal Move Check Function**- Checks whether the move is legal or not based on user input
- **King in Check Function-** Checks whether the king is in check
- **King in Checkmate Function-** Checks whether a checkmate has occurred against the king
- **En Passant Function-** Handles the En Passant move
- **Castling Function-** Handles castling
- **Capture Function-** removes a captured piece from the board
- **Promotion Function-** promotes an ant once it reaches the opposing end of the board
- **Ant Eating Function-** Handles Ant Eating move by Anteater
- **King in Stalemate Function** - Checks whether a stalemate has occurred (player has no legal moves, but the King is not in check). The game ends as a draw.

File I/O Module Interface:

- **Save Function-** Saves the current game and board setup
- **Load Function-** Reloads the previous game and board setup
- **Log File Function-** Records moves made during the game into a log file

Main Module Interface:

- **Undo Function-** Reverses the most recent move
- **Hint Function-** Gives the player the best or a good move to make
- **Pause Function-** Pauses the game
- **Start Function-** Starts the game
- **Difficulty Selection Function-** Selects difficulty for the computer

AI Module Interface:

- **Compute All Legal Moves Function-** Determines all legal moves for the computer
- **Select Best Move Function-** Determines the best move for the computer to make

1.4 Overall program control flow

1. Player Starts the Game:
    a. The Main Menu is displayed, and the player presses the play button to initiate the game.
2. Game Settings are Loaded:
    a. The Rules and different Chess Moves are explained to the player. The colors white and black are assigned to the player/AI. The initial chessboard is displayed at the start.

3. Chess Game is Played: (Loop)
    a. A move is made by White first, and its validity is determined
        i. Invalid move prompts the player/AI to try again.
        ii. Valid move continues the game normally
            1. If this move results in a checkmate, then the White side wins, and the game ends
            2. If this move results in a stalemate, the game ends as a draw.
    b. *Continue Normally:* The chessboard is updated to display the move made.
    c. Now it's the Black side's turn to make a move, and its validity is checked.
        i. Invalid move prompts the player/AI to try again.
        ii. Valid move continues the game normally
            1. If this move results in a checkmate, then the Black side wins, and the game ends
            2. If this move results in a stalemate, the game ends as a draw.
    d. *Continue Normally:* Display an updated chessboard.
    e. Loop until the game ends or the player exits voluntarily.

2. Installation

2.1 System requirements and compatibility

The following environment is required to build and run Anteater Chess:

- Operating System: Linux (Ubuntu 20.04+ recommended). Tested on the UCI course servers: bondi, laguna, crystalcove, zuma, balboa, doheny.
- Compiler: GCC 9.0 or later. The program is written in ANSI C (C99 standard).
- Build System: GNU Make 4.0 or later.
- Graphics Library: SDL2 (libsdl2-dev) for the graphical user interface. If SDL2 is not available, the program falls back to the ASCII terminal display.
- Disk Space: At least 50 MB for source, build artifacts, and game assets.
- Memory: At least 64 MB RAM.
- Terminal: Any ANSI-compatible terminal emulator for the text-based display mode.

To verify GCC and Make are available on the UCI servers:

gcc --version

make --version

2.2 Setup and configuration

Follow these steps to set up the project on development machine or the UCI servers:

1-Log in to your team account on any UCI course server (e.g., bondi):

```
ssh teamN@bondi.ics.uci.edu
```

2-Navigate to your team directory and clone or copy the source files:

```
cd ~
mkdir -p chess
cd chess
```

3-Ensure the directory structure matches the expected layout:

```
chess/
  src/         (all .c source files)
  include/     (all .h header files)
  assets/      (board graphics, piece images)
  doc/         (this document and the user manual)
  Makefile
```

4-(Optional) Install SDL2 on a personal Linux machine if developing locally:

```
sudo apt-get install libsdl2-dev libsdl2-image-dev
```

No additional environment variables or configuration files are required beyond the Makefile.

2.3 Building, compilation, and installation

All build commands are run from the top-level chess/ directory.

Build the program

```
cd chess
```

```
make
```

This compiles all source files in src/ using gcc -std=c99 -Wall -Wextra and produces the executable bin/chess.

Run the program

```
./bin/chess
```

Run automated tests

make test

Executes the test suite in test/ and reports pass/fail for each module.

Clean build artifacts

make clean

Removes all object files and the compiled binary.

Uninstalling

To remove the entire project, delete the chess/ directory:

rm -rf chess/

3. Documentation of Packages, Modules, and Interfaces

3.1 Data Structures:

Piece Structure:

Conceptual Code Segment:

```
struct Piece
{
     enum PieceType Piece
     enum PieceColor Color
}
```

Description: This data structure will be used in declaring the various pieces on the board. PieceType will represent the different possible moves and limitations for a rook, queen, knight, etc. PieceColor is used to differentiate which pieces belong to each player, black or white.

Example Code: Declaration

```
struct Piece White Queen {Queen, White};
```

Piece Type Enumeration:

Conceptual Code Segment:

```
enum PieceType { KING, QUEEN, ROOK, BISHOP, KNIGHT, PAWN,
ANTEATER };
```

Description: Enumerates every piece type. ANTEATER is the custom piece introduced in Anteater Chess.

Piece Color Enumeration:

Conceptual Code Segment:

```
enum PieceColor { WHITE, BLACK };
```

Description: Enumerates the two possible piece owners.

Chess Board:

Conceptual Code Segment:

```
struct Piece *board[8][10]
```

Description: The chessboard will be initialized as a 2D array where the 'squares of the board' are pointers to a Piece structure, rook, knight, queen, etc. Using a pointer allows for empty spaces (NULL) to be taken into account easily. Instead of copying the data structures when moving pieces each time, the memory address is the only thing needed to be updated. The size of the array takes into account the extra space needed for the anteater pieces.

Position Structure:

Conceptual Code Segment:

```
Struct Pos
{
     int rank
     int file
}
```

Description: The position structure contains the integers rank and file. Rank represents the rows of the chessboard numbered 1-8, while files represent the columns labeled A-J.

Move Structure:

Conceptual Code Segment:

```
Struct Move
{
struct Pos from;
struct Pos to;
}
```

Description: The Move data structure consists of the instances Pos1 and Pos2. Pos1 is the coordinate of the starting position. Additionally, Pos2 is the coordinate for where the piece goes next. This structure is dependent on the previous Pos structure.

Move History (MoveLog):

Conceptual Code Segment:

```
#define MAX_MOVES 512
```

```
struct MoveLog {
struct Move moves[MAX_MOVES];
int count;
};
```
Description: Stores the full history of moves made during the game. Required for Undo, log file output,
en passant detection (must check the previous move), and AI evaluation.

Game State:
Conceptual Code Segment:
```
struct GameState {
struct Piece *board[8][10];
enum PieceColor current_turn;
struct MoveLog history;
int white_castle_k;
int white_castle_q;
int black_castle_k;
int black_castle_q;
struct Pos en_passant_target;
};
```
Description: The master record of the entire game at any moment. Save/load, undo, and AI all operate on a GameState.

3.2 Description of functions and parameters

Board Display Function: (Board Module)
Description:
This function displays the chessboard initially and after updating.
Function Signature:
```
void display_board(struct Piece *board[8][10] )
```

Menu Function: (Menu/UI Module)
Description: During a game, if the player wishes to pause the game, a menu will appear prompting the user to select what they wish to do next.
Function Signature:
```
void paused_menu(void)
```

Look Up a Piece Function: (Move Module)
Description: Checks which piece is located at a given board position. It is used to validate moves and whether a piece occupies a square

Function Signature:

```
struct piece lookup_piece( struct Piece *board[8][10],
struct Pos position )
```

Put a Given Piece onto a Given Position Function: (Board Module)

Description:

This function assigns a piece to its corresponding spot on the chessboard initially.

Function Signature:

```
void assign_piece (struct Piece *board[8][10], struct Piece
*piece, struct Pos position)
```

Piece Move Function: (Move Module)

Description:

This function moves a piece from its starting position to a desired position.

Function Signature:

```
void move_function( struct Piece *board[8][10], struct Move move )
```

Compute all Reachable Moves Function: (Move Module)

Description: Determines all places a piece can reach from its given movement pattern

Function Signature:

```
void compute_reachable_moves( struct Piece *board[8][10],
struct Pos position,struct Move *out, int *count );
```

Compute all Legal Moves: (Move Module)

Description: Determines the moves a piece can make given the rules of Anteater chess. Filters reachable moves to exclude any that leave the player's own King in check.

Function Signature:

```
void legalmoves_function(struct GameState *gs, struct Pos
position, struct Move*out, int *count);
```

• gs — current game state (includes board and move history).
• position — position of the piece to evaluate.
• out — array to receive legal moves.
• count — receives the number of legal moves found.

King in Check Function: (Rules Module)

Description:

If a king is in check, then the function will return true and false if a king is not in check.

Function Signature:

```
bool king_in_check( struct Piece *board[8][10] )
```

King in Checkmate Function: (Rules Module)
Description:
If the player on one side does not have any other possible moves for the king to escape, then the function will return true for checkmate. The function will return false if the king is still able to move to escape.
Function Signature:

```
bool king_in_checkmate( struct Piece *board[8][10] )
```

King in Stalemate Function (Rules Module)
Description:
Returns true (1) if the player of the given color has no legal moves but their King is NOT in check. The game ends as a draw.
Function Signature:

```
int king_in_stalemate(struct GameState *gs, enum PieceColor color);
```

• gs — current game state.
• color — the side to test.

En Passant Function: (Rules Module)
Description:
Handles En Passant move.
Function Signature:

```
void enpassant_function( struct Piece *board[8][10], struct Move move)
```

Castling Function: (Rules Module)
Description:
Handles castling move. Moves both the King and Rook to their castled positions and updates castling-rights flags in the game state.
Function Signature:

```
void castling_function(struct GameState *gs, struct Move move);
```

• gs — current game state (castling flags updated in place).
• move — the King's two-square move toward the Rook.

Capture Function: (Rules Module)
Description:
The captured piece will be removed from the board…

Function Signature:

```
     void capture ( struct Piece *board[8][10], struct Pos
position )
```

Promotion Function: (Rules Module)

Description:

Once an Ant reaches the edge of the board: White (A1-J1) and Black (A8-J8), then the Ant can promote to any piece.

Function Signature:

```
     void promote_function( struct Piece *board[8][10], struct
Pos position )
```

Ant Eating Function: (Rules Module)

Description:

Handles the Ant Eating move

Function Signature:

```
     void ant_eating( struct Piece *board[8][10], struct Move
move )
```

Save Function: (File I/O Module)

Description:

Saves current game and board setup to a file that can be loaded later

Function Signature:

```
     int save_function(struct GameState *gs, const char
*filename);
```

• gs — current game state to save.
• filename — path to the output file.
• Returns 0 on success, -1 on file error.

Load Function: (File I/O Module)

Description:

Loads a previously saved game and board setup from a file

Function Signature:

```
     int load_function(struct GameState *gs, const char
     *filename);
```

Log File Function: (File I/O Module)

Description:

Records the moves made during the game

Function Signature:

```
int logfile_function(struct Move move, int turn_number,
enum PieceColor color, const char *filename);
```

Undo Function:

Description:

Reverses the most recent move played during the game

Function Signature:

```
void undo_function(struct GameState *gs)
```

Hint Function:

Description:

Gives the player a hint for the best move to make

Function Signature:

```
void hint_function(struct GameState *gs)
```

Pause Function:

Description:

Pauses the game

Function Signature:

```
void pause_function( void )
```

Start Function:

Description:

Starts the game

Function Signature:

```
void start_function( void )
```

Difficulty Selection Function

Description:

Function selects the level of difficulty for the computer against the player . Level 1 = beginner (depth 1), Level 2 = intermediate (depth 3), Level 3 = expert (depth 5).

Function Signature:

```
void difficultyselect_function(int level);
```

Compute all Legal Moves Function: (AI Module)

Description:

Determines all legal moves for both the computer and player

Function Signature:

```
    void ai_legalmoves_function(struct GameState *gs, enum
PieceColor color, struct Move *out, int *count);
```

Select Best Move Function: (AI Module)
Description:
Determines the best move for the computer to make
Function Signature:

```
    struct Move bestmove_function(struct GameState *gs, enum
PieceColor color, int depth);
```

Legal Move Check Function:
Description:
Checks whether a move is legal based off user input while following the rules of Anteater chess
Function Signature:

```
    void legalmovecheck_function( struct Piece *board[8][10],
struct Move move )
```

3.3 Description of input and output formats

The syntax or format of a move entered by the user:

- The user will be prompted with two questions: which piece they wish to move and where they'd like to move it. The user input for these positions will use the format of 'A2', where the file letter is stated first, followed by the rank number.

The syntax or format of a move recorded in the log file:

- Each line records one half-move in the following format:
- <turn>.<color> <from_file><from_rank>-<to_file><to_rank>
- Example:
- 1.W F2-F4
- 1.B E7-E5
- 2.W D1-H5
- 2.B B8-C6
- turn — the full-move number (increments after Black moves).
- color — W for White, B for Black.
- Log file is named chess_log.txt, created fresh at the start of each game.

4. Development Plan and Timeline

4.1 Partitioning of Tasks and Timeline

Week	Dates	Tasks
1-2	Mar 31 – Apr 13	User Manual, Software Spec documents. Core data structures (Piece, Board, Pos, Move, GameState). Makefile skeleton.
3	Apr 14 – Apr 16	Board module (init, display, lookup, assign). Move module (apply_move, compute_reachable_moves). Text-based UI. Basic game loop in Main.
4	Apr 17 – Apr 20	Rules module (check, checkmate, stalemate, castling, en passant, promotion, ant eating). File I/O (save, load, log). Alpha release.
5	Apr 21 – Apr 24	AI module (minimax + alpha-beta). SDL2 GUI. Undo, Hint, Difficulty. Integration testing.
6	Apr 25 – Apr 27	Bug fixes, tournament preparation, final release (Chess_V1.0). Documentation cleanup.

4.2 Team member responsibilities

Team member	Primary Responsibilities
Akshithi Suresh	Core data structures (Piece, Board, Pos, Move, GameState, MoveLog) with code snippets. Board module (init_board, display_board, lookup_piece, assign_piece). Undo function (relies directly on MoveLog). Documentation lead and Makefile.
Jolynn Quang	Rules module standard move validation: is_legal_move, compute_legal_moves, is_king_in_check, is_checkmate, is_stalemate. Castling (handle_castling). Shared module testing.
Jasmine Kong	Rules module special pawn moves: en passant (handle_en_passant), promotion (handle_promotion), ant-eating (handle_ant_eating), capture. Move module (apply_move, compute_reachable_moves). Shared module testing.
Troy Reindl	AI module minimax algorithm with alpha-beta pruning, board evaluation heuristic, ai_compute_all_legal_moves, ai_select_best_move, set_difficulty. Hint function (thin wrapper over AI). Shared module testing.
Ayda Ahmadian	File I/O module (save_game, load_game, log_move). Main module game loop, show_main_menu, show_pause_menu, get_user_move, pause/start/quit control flow. Shared module

	testing.
Trisha Satyavrat	SDL2 graphical user interface (board rendering, piece graphics, click/drag input). Integration testing and end-to-end test suite. Tournament preparation and final release packaging (Chess_V1.0.tar.gz, Chess_V1.0_src.tar.gz).

Troubleshooting and Error Messages

Error/ Symptom	**Cause/ Resolution**
make: command not found	GNU Make is not installed or not in PATH. On UCI servers, run: module load make
SDL2 not found during compilation	SDL2 development headers missing. Run: sudo apt-get install libsdl2-dev. Alternatively, compile with make TEXT_ONLY=1 to use the ASCII display.
Error: Invalid move format	The entered coordinate does not match the expected format (e.g., 'F2 F4'). Ensure file is A–J and rank is 1–8, separated by a space.
Error: Illegal move	The move violates a chess rule or leaves the King in check. Try a different move or use the Hint function.
Segmentation fault on load	The save file may be corrupt or from an incompatible version. Delete the file and start a new game.
AI move takes longer than 1 minute	Search depth is too high for the current position. Reduce difficulty with the set_difficulty function (use level 1 or 2).

Board displays incorrectly in terminal	The terminal does not support ANSI escape codes. Set TERM=xterm-256color or use the GUI mode.

Index

Copyright

References

1. FIDE Laws of Chess. Fédération Internationale des Échecs (FIDE). Available at: https://www.fide.com/FIDE/handbook/LawsOfChess.pdf

2. Dömer, R. EECS 22L: Software Engineering Project in C Language, Lectures 1–2. University of California, Irvine, 2026.

3. SDL2 Documentation. Simple DirectMedia Layer. Available at: https://wiki.libsdl.org/SDL2/FrontPage

4. Knuth, D. E., and Moore, R. W. (1975). An Analysis of Alpha-Beta Pruning. Artificial Intelligence, 6(4), 293–326.